心音（PCG）分类算法——基于深度学习-树莓派的心脏疾病听诊

知错就改
code and struggle for life
关注他

收录于 • 本科项目 ›

136 人赞同了该文章 ›

最近我们做了一个基于PCG信号+的心音诊断项目。项目大致内容：我们开发了一套心脏疾病诊断算法，并将该算法部署于树莓派zero+上，制作如下诊断仪：



未连接听诊器的诊断仪

我们先来看一下效果：

B站视频链接：

GitHub链接：

这篇文章将围绕我们使用的**数据集介绍，数据预处理，数据集融合**，**特征提取**，**神经网络结构**，**硬件部署**，**美化**这七个方面对心脏疾病诊断仪进行详细展开介绍。

一、数据集介绍

我们共使用了4份数据集，我们简单描述四份数据集的组成与结构。我们会在后面详细介绍如何合并与融合这四份数据。

1.1 Database1

这份数据集由Yaseen[2018]提供，其中包含5种类型的心音，概括如下表所示：

文件夹	心音状态（疾病名称）	音频数
N	正常	200
AS	主动脉瓣狭窄	200
MS	二尖瓣狭窄	200
MR	二尖瓣反流	200
MVP	二尖瓣脱垂	200

数据集一的链接：

1.2 Database2

这份数据是2016 PhysioNet/CinC Challenge所用数据集，概括如下表所示：

数据集	环境	病人数	病理特征	正常数	异常数
a	家/医院	121	MVP	292	117
b	医院	106	Murmurs	104	386
c	医院	31	MR, AS	24	7
d	医院/实验室	38	None	28	27
e	医院/实验室	356	Murmurs	183	1958
f	医院	112	None	34	80

数据集二的链接：

1.3 Database3

这份数据集是Kaggle2016年的一个关于心音的竞赛数据，概括如下表所示：

数据集	正常音频数	异常音频数	额外音频数	人造音频数
A	31	34	19	40
B	200	66	46	0

数据集三的链接：

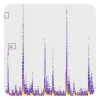
https://www.kaggle.com/kinguistics/heartbeat-sounds
www.kaggle.com/kinguistics/heartbeat-sounds

1.4 Database4

这份数据集也是一个竞赛用的数据集，概括如下表所示：

数据集	正常音频数	异常音频数	额外音频数	人造音频数
A	31	34	19	40
B	320	95	46	195

数据集四的链接：



Classifying Heart Sounds Challenge
www.peterjbentley.com/heartchallenge/

二、数据预处理

我们知道，不同数据集的音频数据制作标准差异很大，这种差异包括：音频采样率，通道数，长短，降噪法等等。我们在融合数据集之前需要最大可能地统一这些标准。因此，我们需要对所有的音频文件进行数据预处理。

下面，我们以Database2中training-a的第一条音频数据为例，介绍我们数据预处理法：

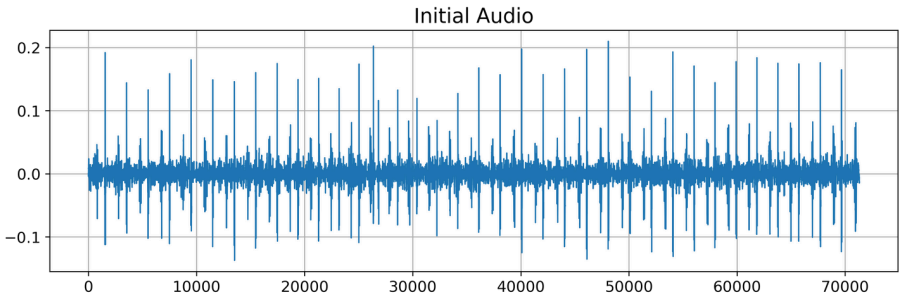
2.1 读取音频

python中读取音频文件的方法非常多，这里我们选择调用librosa包，并可视化音频。实现代码如下：

```
import librosa
import matplotlib.pyplot as plt


def plot_signal(audio_data, title=None):
    plt.figure(figsize=(12, 3.5), dpi=300)
    plt.plot(audio_data, linewidth=1)
    plt.title(title,fontsize = 16)
    plt.tick_params(labelsize=12)
    plt.grid()
    plt.show()
```

```
audio_path = '../databases/database2/training-a/a0001.wav'
audio_data, fs = librosa.load(audio_path, sr=None)
plot_signal(audio_data, title='Initial Audio')
```



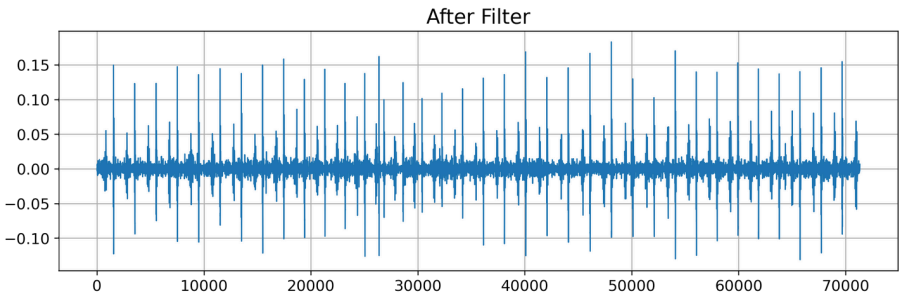
2.2 数字滤波

由于音频在制作时不可避免地会保存一部分噪声，我们需对音频文件进行数字滤波，旨在滤除高频噪声以及直流噪声，同时尽可能保留心音信号。我们把音频送入二阶25-400hz的巴特沃斯中值滤波器，并可视化音频。实现代码如下：

```
from scipy import signal


def band_pass_filter(original_signal, order, fc1,fc2, fs):
    '''
    中值滤波器
    :param original_signal: 音频数据
    :param order: 滤波器阶数
    :param fc1: 截止频率
    :param fc2: 截止频率
    :param fs: 音频采样率
    :return: 滤波后的音频数据
    '''
    b, a = signal.butter(N=order, Wn=[2*fc1/fs,2*fc2/fs], btype='bandpass')
    new_signal = signal.lfilter(b, a, original_signal)
    return new_signal


audio_data = band_pass_filter(audio_data, 2, 25, 400, fs)
plot_signal(audio_data, title='After Filter')
```



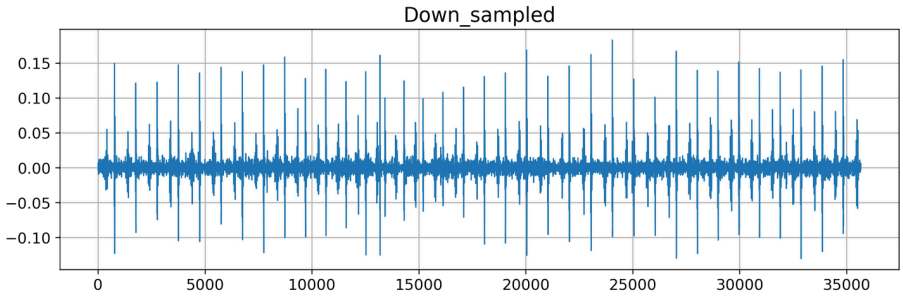
2.3 下采样

为了降低模型的计算量，我们对所有的音频信号进行下采样，考虑到我们已经对音频进行了25-400hz的中值滤波，根据奈奎斯特采样定律，我们把信号下采样到1000hz。这里我们调用samplerate包来完成这项工作，并可视化音频。下面给出实现代码：

大家都在搜 换一换
警方称问界汽车事故司机… 567 万 热
中土网民爆发大规模互怼 416 万 热
女子地铁站看书5小时被… 304 万 热
解放军黄海持续47天实弹… 294 万 热
西藏地震16人遇难 293 万 热
「死了么」App公布收费… 290 万 热
留美女博士因答辩失利投毒 288 万 热
泽连斯基称保留75%领土… 287 万
子祺事变 273 万 热
英伟达称消费级显卡超出… 269 万



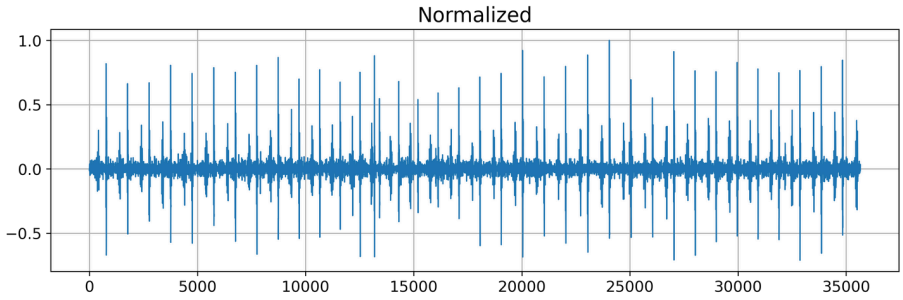
```
# 下采样
down_sample_audio_data = samplerate.resample(audio_data.T, 1000 / fs, converter_t
plot_signal(down_sample_audio_data, title='Down_sampled')
```



2.4 归一化

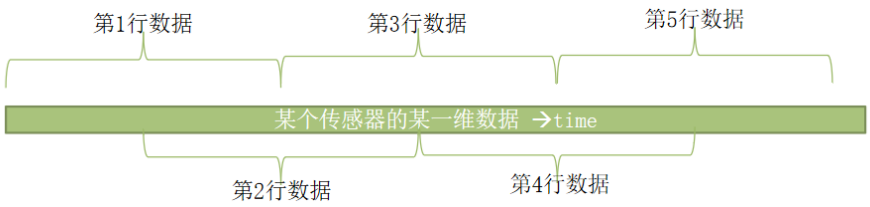
由于不同数据集中的音频文件尺度差异较大，我们对所有的音频信号进行归一化，使其范围在[-1,1]区间内，并可视化音频。实现代码如下：

```
# 归一化
import numpy as np
down_sample_audio_data = down_sample_audio_data / np.max(np.abs(down_sample_audio
plot_signal(down_sample_audio_data, title='Normalized')
```



2.5 切割音频

为了尽可能多利用已有的数据集，我们对较长的音频进行切割。我们以2.5s为单位对音频进行切割。同时，为了尽可能多获取一些信息，我们选择了带有50%的overlap进行切割。overlap的定义可以参考下图：



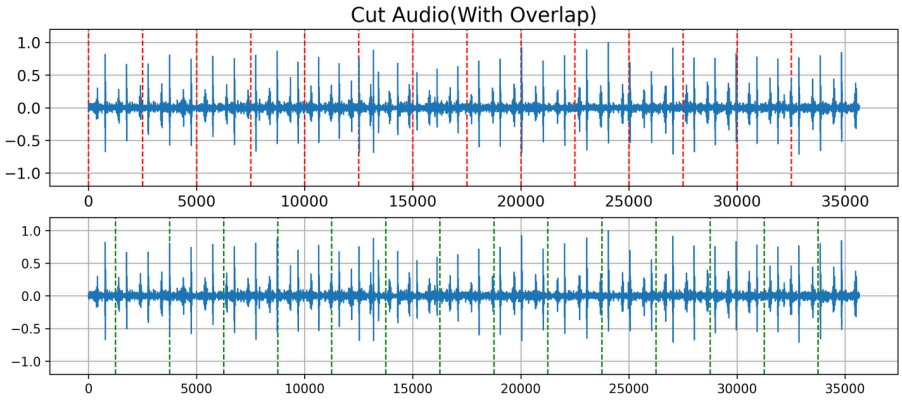
我们将这一过程可视化，实现代码如下：

```
total_num = len(down_sample_audio_data)/(2500)# 计算切割次数
fig = plt.figure(figsize=(12, 5), dpi=300)

ax1 = fig.add_subplot(2,1,1)
plt.plot(down_sample_audio_data,l
plt.title('Cut Audio(With Overlap
```

```
for j in range(int(total_num)):
    plt.vlines(j*2500, -1.2, 1.2, color="red",linestyle='--',linewidth=1.1)

ax2 = fig.add_subplot(2, 1, 2)
plt.plot(down_sample_audio_data, linewidth=1)
for j in range(int(total_num)):
    plt.vlines(j*2500 + 1250, -1.2, 1.2, color="green",linestyle='--',linewidth=1
plt.ylim([-1.2,1.2])
plt.grid()
plt.show()
```



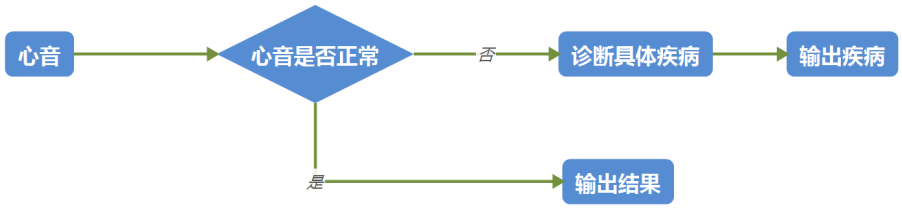
依照以上操作，我们对所有的音频文件进行预处理。

三、融合数据集

预处理工作完成后，我们就可以开始融合数据集了。

我们知道数据集是为了训练模型服务的。那么先来看看我们的需要是什么?

为了能够让使用者得到高效、实用的诊断，我们设计了如下的流程图：



显然，我们需要为了**训练两个不同的模型**，分别是

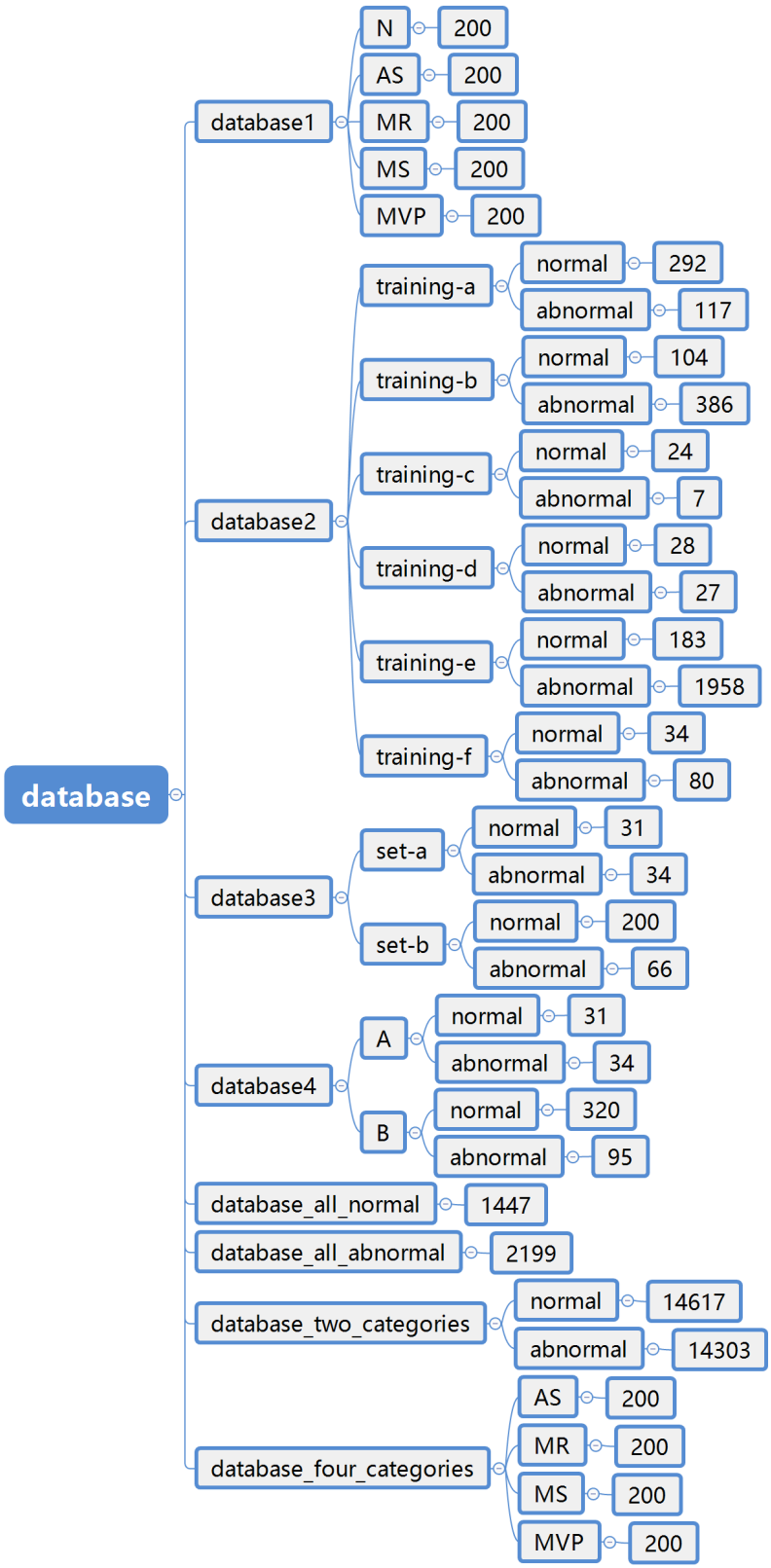
- 二分类模型，判断测试者是否健康
- 四分类模型，识别具体疾病

因此，根据需求，我们需要两份不同的融合数据集

- 第一个模型需要一个二分类数据集
- 第二个模型需要一个四分类数据集

于是，我们把所有的数据集组成如下两套数据：

- 数据集一(database_two_categories)包含所有的正常心音与不正常心音
- 数据集二(database_four_categories)仅包含AS，MS，MR，MVP异常四类心音



四、特征提取

整理得到了数据集，那么我们开始对所有的数据进行特征提取。

> **Ali Mohammad Alqudah** 等人的研究已证明，现代数字信号处理领域中的高阶谱分析方法提取出的特征明显优于**短时傅里叶变换**、**小波变换**等**低阶特征提取方法**的结果。此外，高阶谱方法中的二阶谱法使用最为广泛，它可以很好地抑制信号中的相位关系，检测与量化非高斯信号的相位耦合，常被用于

二阶谱分析法可用如下公示表示：

$$S_2^x(\omega_1,\omega_2)=\sum_{\tau_1=-\infty}^{+\infty}\sum_{\tau=-\infty}^{+\infty}c_3^x(\tau_1,\tau_2)\exp\{-j(\omega_1\tau_1+\omega_2\tau_2)\}$$

$$c_3^x(\tau_1,\tau_2)=E\{x(n)x(n+\tau_1)x(n+\tau_2)\}$$

其中，$S_2^x(\omega_1,\omega_2)$ 为二阶傅里叶变换，$c_3^x(\tau_1,\tau_2)$ 为三阶累积量

由公式我们不难看出，生成的二阶谱是一个二维矩阵(256×256)。于是，我们根据此矩阵分别绘制其等高线图$(256\times256\times3)$与热力图$(256\times256\times3)$，加上原本的二阶谱，共生成了3种特征。

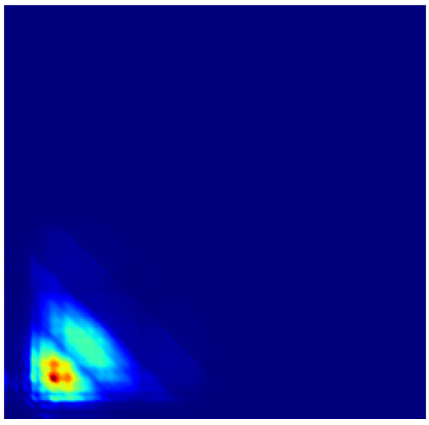
关于二阶谱提取的代码，github上有很多，但是几乎都不能运行，最终我找打了为一个能用的，把链接放在这里：

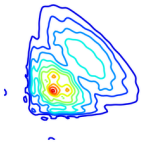
https://github.com/trichter/polycoherence
github.com/trichter/polycoherence

我们参考这个项目，适当调整代码。下面我们仍以Database2中training-a中的第二条音频为例（该代码可以衔接在上面的数据预处理过程中后，作为一个完整的单个音频处理方案），给出我们特征提取实现代码：

```
# 特征提取
from extract_bispectrum import polycoherence,plot_polycoherence

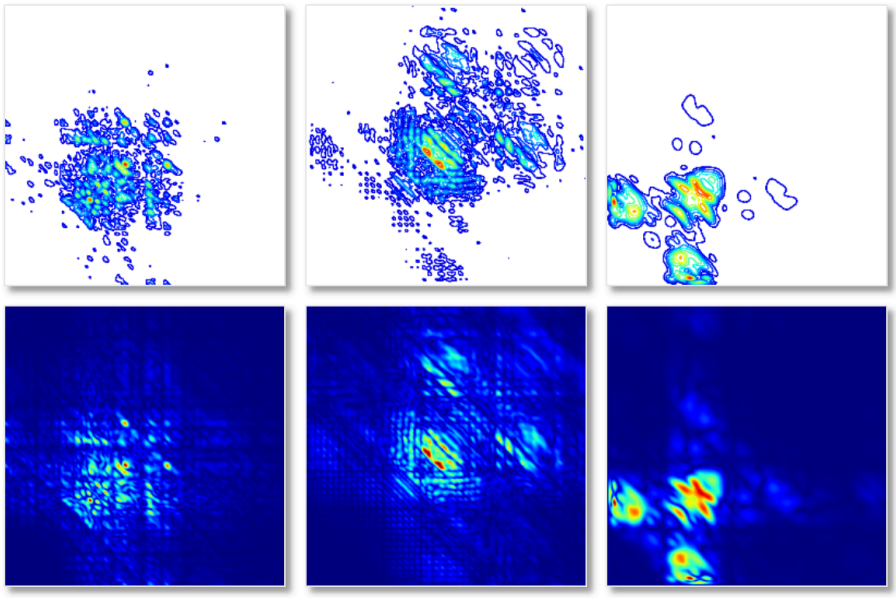

ex_audio_data = down_sample_audio_data[:2500]
freq1, freq2, bi_spectrum = polycoherence(
       ex_audio_data,
       nfft=1024,
       nperseg=256,
       noverlap = 100,
       fs = 1000,
       norm=None)
bi_spectrum = np.array(abs(bi_spectrum))  # calculate bi_spectrum
bi_spectrum = 255 * (bi_spectrum - np.min(bi_spectrum)) / (np.max(bi_spectrum) - 
plot_polycoherence(freq1, freq2, bi_spectrum)
# 修改尺寸以便于投入神经网络
bi_spectrum = bi_spectrum.reshape((256, 256, 1))
```





https://drive.google.com/file/d/10ieaL7CRcnD-Ja0MpJnm8SHQpr1l6ilG/view?usp=sharing
drive.google.com/file/d/10ieaL7CRcnD-Ja0MpJnm8SHQ...

进而，我们再给出其他三种不同类的音频的二阶谱特征图，如下所示：



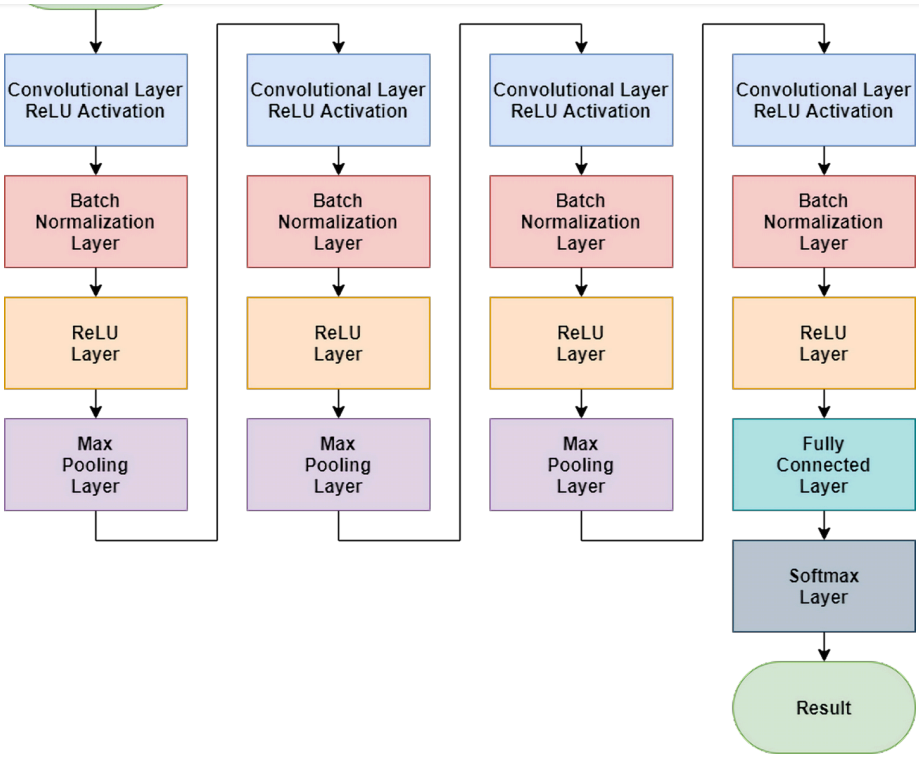
上图非常的直观，我们看出不同类别的心音信号的二阶谱图差异是很大。这将大大帮助我们进行心音分类。我们在实验中，也证实了这是一种有效的特征提取之法。

特别地，在项目实践中，我们发现使用二维矩阵形式的二阶谱与它的等高线图和热力图所得到的结果相差不大。因此，为了减小模型的计算量，尽量提高速度，我们使用的特征就是二阶谱本身。

五、网络结构

我们所训练的两个模型，采用了同一种神经网络结构，我们下面对该结构进行介绍。

我们采用改进的AOCT卷积神经网络[+]。该卷积神经网络可以非常好得提取出二阶谱中的特征，从而完成进行分类任务，非常适用于我们的需求。网络中共使用了4个以卷积层、BN层、激活函数为整体的卷积块，再将结果传入全连接层中得到最终结果。其具体结构如下图所示：



我们利用keras实现该卷积神经网络码，并给出代码：

```
from tesorflow import keras


def build_model():
    inputdata = keras.Input(shape=(256, 256, 1))

    final = keras.layers.Conv2D(32, (3, 3), padding="same",activation='relu')(inpu
    final = keras.layers.BatchNormalization()(final)
    final = keras.layers.ReLU()(final)
    final = keras.layers.MaxPooling2D((2, 2), strides=(2, 2))(final)

    final = keras.layers.Conv2D(16, (3, 3), padding="same",activation='relu')(fina
    final = keras.layers.BatchNormalization()(final)
    final = keras.layers.ReLU()(final)
    final = keras.layers.MaxPooling2D((2, 2), strides=(2, 2))(final)

    final = keras.layers.Conv2D(8, (3, 3), padding="same",activation='relu')(final
    final = keras.layers.BatchNormalization()(final)
    final = keras.layers.ReLU()(final)
    final = keras.layers.MaxPooling2D((2, 2), strides=(2, 2))(final)

    final = keras.layers.Conv2D(16, (3, 3), padding="same",activation='relu')(fina
    final = keras.layers.BatchNormalization()(final)
    final = keras.layers.ReLU()(final)

    final = keras.layers.Flatten()(final)
    final = keras.layers.Dense(2)(final)# 二分类时为2，四分类时为4
    # final = keras.layers.BatchNormalization()(final)
    final = keras.layers.Softmax()(final)

    model = keras.Model(inputs=inputdata, outputs=final)
    optimizer = keras.optimizers.Adam(
        lr=0.001, beta_1=0.9, beta_2=0.999, epsilon=None)
    model.compile(loss='sparse_ca
                  optimizer=optim
```

我们使用深度学习服务器进行训练：

```
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 418.87.00    Driver Version: 418.87.00    CUDA Version: 10.1     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|===============================+======================+======================|
|   0  Tesla V100-PCIE...  Off  | 00000000:18:00.0 Off |                    0 |
| N/A   61C    P0   130W / 250W |  31400MiB / 32480MiB |     84%      Default |
+-------------------------------+----------------------+----------------------+
|   1  Tesla V100-PCIE...  Off  | 00000000:5E:00.0 Off |                    0 |
| N/A   31C    P0    36W / 250W |    318MiB / 32480MiB |      0%      Default |
+-------------------------------+----------------------+----------------------+
|   2  Tesla V100-PCIE...  Off  | 00000000:AF:00.0 Off |                    0 |
| N/A   32C    P0    35W / 250W |    318MiB / 32480MiB |      0%      Default |
+-------------------------------+----------------------+----------------------+
|   3  Tesla V100-PCIE...  Off  | 00000000:D8:00.0 Off |                    0 |
| N/A   33C    P0    37W / 250W |    318MiB / 32480MiB |      0%      Default |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| Processes:                                                       GPU Memory |
|  GPU       PID   Type   Process name                             Usage      |
|=============================================================================|
|    0     66720      C   python                                   31389MiB |
|    1     66720      C   python                                     307MiB |
|    2     66720      C   python                                     307MiB |
|    3     66720      C   python                                     307MiB |
+-----------------------------------------------------------------------------+
```

5.1 模型一（2分类）

该模型每一个卷积层的卷积核数量都是前面给出代码中的两倍（不重复给出代码了）。我们将数据集database_two_categories划分为25000个训练数据，3000个测试数据，把数据送入网络，给出训练该模型的代码：

```python
import numpy as np
from model.model_2_categories import build_model
from extract_bispectrum import get_bi_spectrum


def cal_acc(label,prediction):
    num=0
    N = len(label)
    for i in range(len(label)):
        pred = np.argmax(prediction[i])
        if label[i]==pred:
            num+=1
    return num/N


def shuffle_data_label(data, label):
    state = np.random.get_state()
    np.random.shuffle(data)
    np.random.set_state(state)
    np.random.shuffle(label)
    return data,label


def generate_label(num=14000):
    label = []
    for j in range(2):
        for i in range(num):
            label.append(j)
    return np.array(label)
```

```
    np.random.seed(0)
    file_folder = 'databases/database_two_categories'
    class_name = ['n','abn']
    dataset = get_bi_spectrum(file_folder, class_name,14000)
    label = generate_label()
    img_data,label = shuffle_data_label(dataset,label)
    train_data = img_data[:25000,:,:]
    test_data = img_data[25000:,:,:]
    train_label = label[:25000]
    test_label = label[25000:]
    # print(train_data.shape)
    num_epochs = 60
    model = build_model()
    history = model.fit(train_data, train_label,
                        epochs=num_epochs,
                        batch_size=64,
                        # validation_split=0.2,
                        verbose=1)
    model.save('model/h5_file/model_2.h5')
    test_predictions = model.predict(test_data)
    # 保存训练结果，便于后续的计算相关
    np.save('result/result_2/pred',test_predictions)
    np.save('result/result_2/label',test_label)
    acc = cal_acc(test_label, test_predictions)
    print(acc)
```

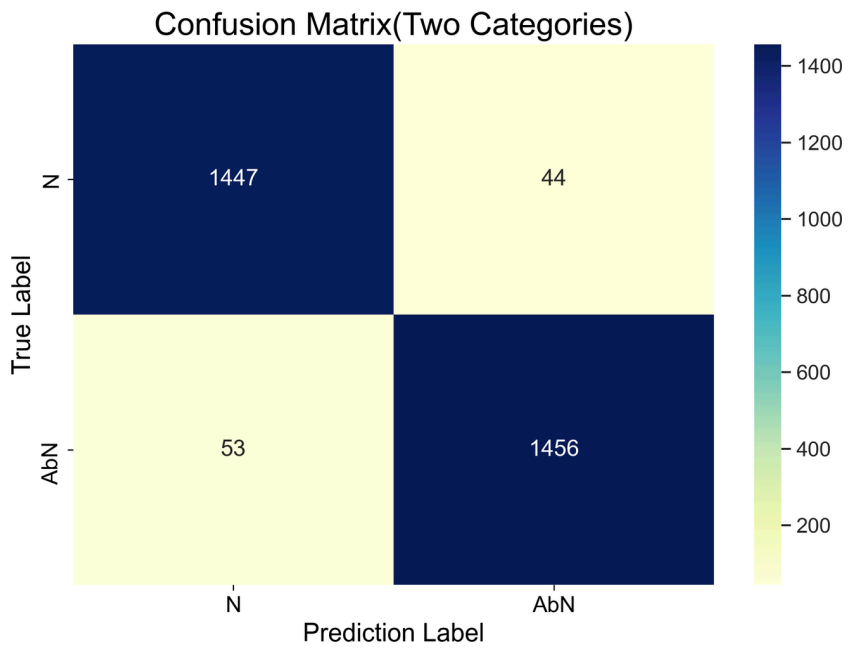
模型在测试集上准确率为96.77%，下面画出混淆矩阵：

```
import numpy as np
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns


def get_result(prediction):
    out = []
    for i in range(len(prediction)):
        out.append(np.argmax(prediction[i]))
    return np.array(out)


label = np.load('label.npy')
pred = np.load('pred.npy')
prediction = get_result(pred)

fig = plt.figure(figsize=(8,6),dpi=300)
ax = fig.add_subplot(111)
conf_mat = confusion_matrix(label, prediction)
sns.set(font_scale=1.2)
sns.heatmap(conf_mat,
            annot=True,
            xticklabels=['N', 'AbN'],
            yticklabels=['N', 'AbN'],
            cmap = plt.cm.YlGnBu
            )
ax.set_title('Confusion Matrix(Two Categories)',fontsize = 20)
ax.set_xlabel('Prediction Label',fontsize = 16)
ax.set_ylabel('True Label',fontsize = 16)
plt.xticks(fontsize=14)
```



5.2 网络二（4分类）

我们将数据集database_four_categories划分为600个训练数据，200个测试数据，给出训练代码：

```
import numpy as np
from model.model_4_categories import build_model
from extract_bispectrum import get_bi_spectrum


def cal_acc(label,prediction):
    num=0
    N = len(label)
    for i in range(len(label)):
        pred = np.argmax(prediction[i])
        if label[i]==pred:
            num+=1
    return num/N


def shuffle_data_label(data, label):
    state = np.random.get_state()
    np.random.shuffle(data)
    np.random.set_state(state)
    np.random.shuffle(label)
    return data,label


def generate_label(num=200):
    label = []
    for j in range(2):
        for i in range(num):
            label.append(j)
    return np.array(label)


if __name__ == '__main__':
    np.random.seed(0)
```

```
    label = generate_label()
    img_data,label = shuffle_data_label(dataset,label)
    train_data = img_data[:600,:,:]
    test_data = img_data[600:,:,:]
    train_label = label[:600]
    test_label = label[600:]
    # print(train_data.shape)
    num_epochs = 500
    model = build_model()
    history = model.fit(train_data, train_label,
                        epochs=num_epochs,
                        batch_size=16,
                        # validation_split=0.2,
                        verbose=1)
    model.save('model/h5_file/model_4.h5')
    test_predictions = model.predict(test_data)
    # 保存训练结果，便于后续的计算相关
    np.save('result/result_4/pred',test_predictions)
    np.save('result/result_4/label',test_label)
    acc = cal_acc(test_label, test_predictions)
    print(acc)
```

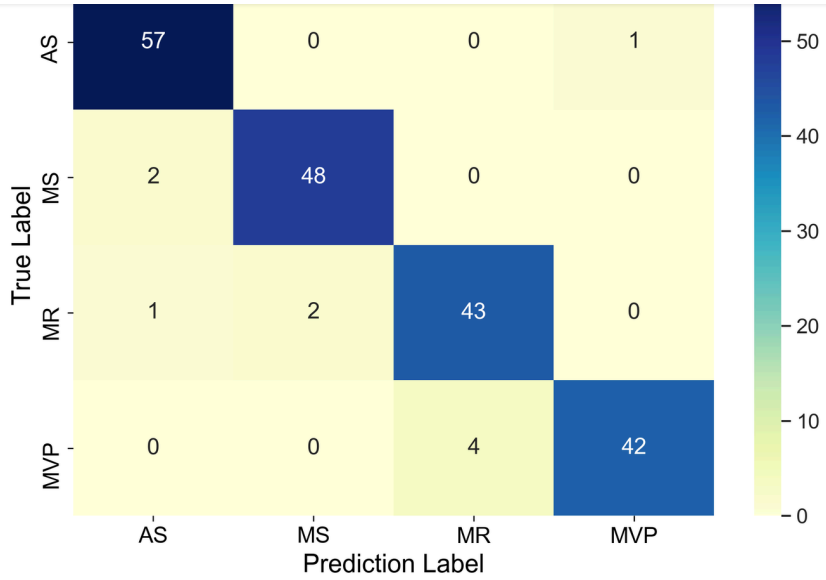
模型在测试集上准确率为97%，下面画出混淆矩阵：

```
import numpy as np
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns


def get_result(prediction):
    out = []
    for i in range(len(prediction)):
        out.append(np.argmax(prediction[i]))
        # print(i,out,prediction[i])
    return np.array(out)


label = np.load('label.npy')
pred = np.load('pred.npy')
prediction = get_result(pred)

fig = plt.figure(figsize=(8,6),dpi=300)
ax = fig.add_subplot(111)
conf_mat = confusion_matrix(label, prediction)
sns.set(font_scale=1.2)
sns.heatmap(conf_mat,
            annot=True,
            xticklabels=['AS', 'MS', 'MR', 'MVP'],
            yticklabels=['AS', 'MS', 'MR', 'MVP'],
            cmap = plt.cm.YlGnBu
            )
ax.set_title('Confusion Matrix(Four Categories)',fontsize = 20)
ax.set_xlabel('Prediction Label',fontsize = 16)
ax.set_ylabel('True Label',fontsize = 16)
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
plt.show()
```

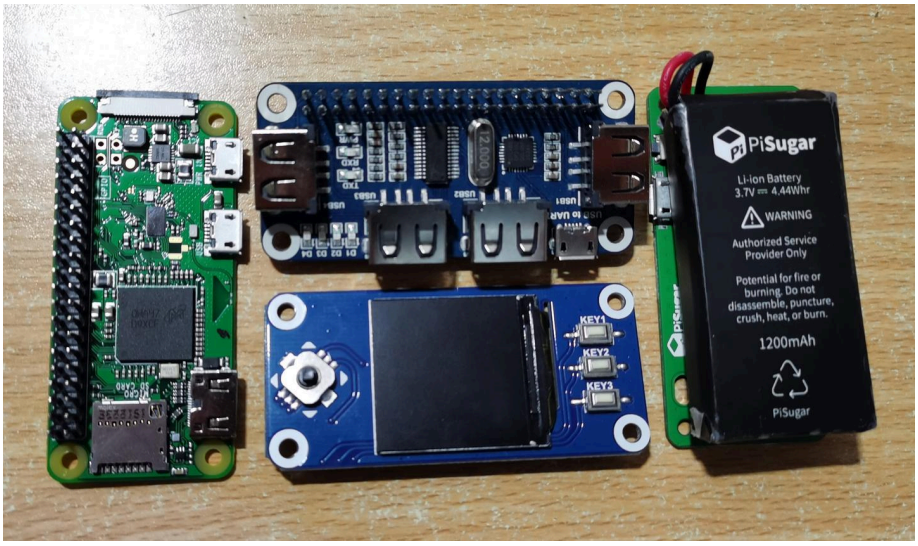


六、硬件部署

6.1 硬件选择

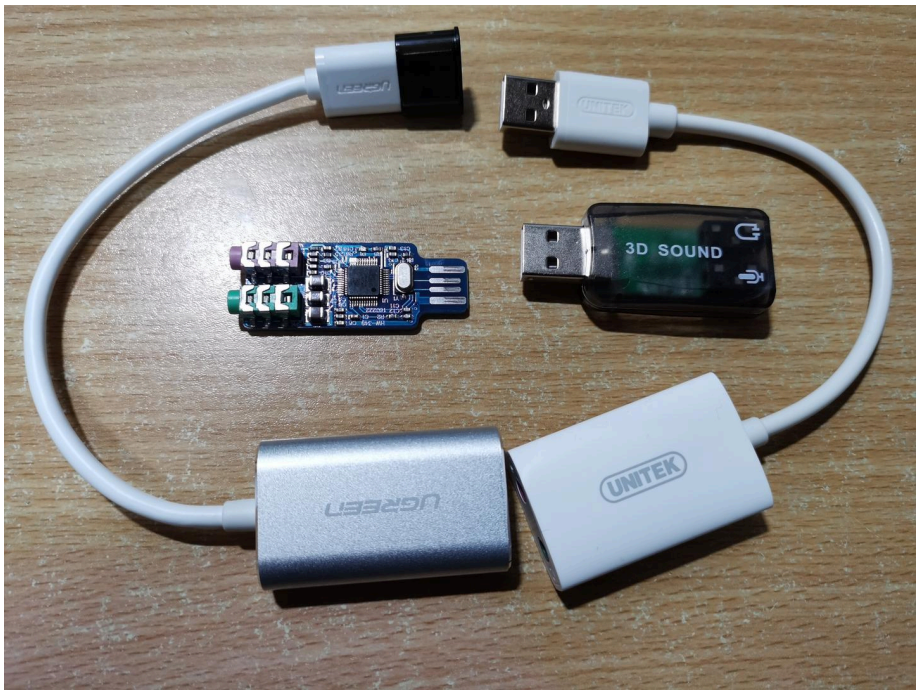
考虑到成本，我们选择将算法部署到树莓派zero。

我们为树莓派购置1200mah电池，微雪USB HUB HAT拓展接口，微雪1.3inch LCD HAT显示屏。



以及3.5mm耳机接口的听诊器搭配外置声卡，使得树莓派可以录音





这里值得一提的是，声卡不是越贵越好，下面两个一眼就是很高大上的家伙价格高，但不适用树莓配，这个很值得注意！！

6.2 算法部署

需要注意的是，在树莓派上，以h5格式保存的网络数据是不能被加载的。因此，我们需要将h5文件转换成tflite文件。我们直接使用**TensorFlow官网**给出的代码进行生成tflite文件。

```
import tensorflow as tf

# Convert the model
converter = tf.lite.TFLiteConverter.from_saved_model(saved_model_dir) # path to t
tflite_model = converter.convert().

# Save the model.
```

我们的项目需要两个模型，因此我们把代码改写成for循环形式

```
import os
import tensorflow as tf


def get_all_filenames(file_dir):
    all_files = [file for file in os.listdir(file_dir)]
    return all_files


all_files = get_all_filenames('h5_file')
for name in all_files:
    model = tf.keras.models.load_model('h5_file/{}'.format(name))
    print(model.summary())
    converter = tf.lite.TFLiteConverter.from_keras_model(model)
    tflite_model = converter.convert()
    # Save the model
    with open('tflite_file/{}.tflite'.format(name[:-3]), 'wb') as f:
        f.write(tflite_model)
```

下一步，自然就是调用tflite格式文件了。同样，我们**参考官方文档**给出的代码：

```
import numpy as np
import tensorflow as tf


# 加载tflite模型
interpreter = tf.lite.Interpreter(model_path="converted_model.tflite")
interpreter.allocate_tensors()

# 获取输入输出的tensors
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

# 此处，请使用你需要的测试数据
input_shape = input_details[0]['shape']
input_data = np.array(np.random.random_sample(input_shape), dtype=np.float32)
interpreter.set_tensor(input_details[0]['index'], input_data)

interpreter.invoke()
output_data = interpreter.get_tensor(output_details[0]['index'])

print(output_data)
```

关于树莓派的初始配置，我们参考

给出的教程。

关于LCD屏幕的配置，我们参考

经过漫长的调试之后，我们终于完成了作品，给出完整版代码：

```
import numpy as np
from PIL import Image


def show_image(im):
    im = im.resize((240, 240), Image.ANTIALIAS)
    with open('/dev/fb1', 'wb') as f:
        img = np.asarray(im)
        pix = np.zeros((240, 240, 2), dtype=np.uint8)
        pix[..., [1]] = np.add(np.bitwise_and(
            img[..., [0]], 0xF8), np.right_shift(img[..., [1]], 5))
        pix[..., [0]] = np.add(np.bitwise_and(np.left_shift(
            img[..., [1]], 3), 0xE0), np.right_shift(img[..., [2]], 3))
        pix = pix.flatten().tobytes()
        f.write(pix)


# Show the init image as fast as possible
im_init = Image.open('/home/pi/Desktop/material/init.png')
show_image(im_init)
im_AS = Image.open('/home/pi/Desktop/material/AS.png')
im_MR = Image.open('/home/pi/Desktop/material/MR.png')
im_MS = Image.open('/home/pi/Desktop/material/MS.png')
im_MVP = Image.open('/home/pi/Desktop/material/MVP.png')
im_wait = Image.open('/home/pi/Desktop/material/wait.png')
im_taking = Image.open('/home/pi/Desktop/material/taking.png')
im_normal = Image.open('/home/pi/Desktop/material/normal.png')
im_abnormal = Image.open('/home/pi/Desktop/material/abnormal.png')
im_welcome = Image.open('/home/pi/Desktop/material/welcome.png')
im_testing = Image.open('/home/pi/Desktop/material/testing.png')

import os
import samplerate
import tensorflow as tf
import soundfile as sf
from scipy import signal
import matplotlib.pyplot as plt
from polycoherence import polycoherence
# import tflite_runtime.interpreter as tflite
import time
import RPi.GPIO as GPIO



def band_pass_filter(original_signal, order, fc1, fc2, fs):
    b, a = signal.butter(
        N=order, Wn=[2 * fc1 / fs, 2 * fc2 / fs], btype='bandpass')
    new_signal = signal.lfilter(b, a, original_signal)
    return new_signal



def plot_signal(audio_data, title=None):
    plt.figure(figsize=(6, 4), dpi=200)
    plt.plot(audio_data, linewidth=1)
    plt.grid()
    plt.savefig('/home/pi/Desktop,
```

```
DC_PIN = 24

KEY_UP_PIN = 6
KEY_DOWN_PIN = 19
KEY_LEFT_PIN = 5
KEY_RIGHT_PIN = 26
KEY_PRESS_PIN = 13

KEY1_PIN = 21
KEY2_PIN = 20
KEY3_PIN = 16

GPIO.setmode(GPIO.BCM)
GPIO.setup(KEY_UP_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(KEY_DOWN_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(KEY_LEFT_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(KEY_RIGHT_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(KEY_PRESS_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(KEY1_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(KEY2_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(KEY3_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP)


show_image(im_welcome)

print('begin')
dataset = np.ones((1, 256, 256, 1))
dataset = dataset.astype('float32')
while 1:

    # If you press key1 and then you pull up
    if not GPIO.input(KEY_PRESS_PIN):
        show_image(im_welcome)
    if not GPIO.input(KEY1_PIN):
        show_image(im_taking)
        os.system('arecord --format S32_LE --rate 2000 -c 2 -d 10 /home/pi/Deskto
        audio_path = '/home/pi/Desktop/sounds/test.wav'
        show_image(im_wait)
        audio_data, fs = sf.read(audio_path)
        audio_data = (audio_data[:, 0] + audio_data[:, 1]) / 2
        # print("原始音频数据点数: ", audio_data.shape, "采样率: ", fs)
        audio_data = band_pass_filter(audio_data, 2, 25, 400, fs)
        down_sample_audio_data = samplerate.resample(
            audio_data.T, 1000 / fs, converter_type='sinc_best').T
        down_sample_audio_data = down_sample_audio_data / np.max(np.abs(down_samp
        # print("现音频数据点数: ", down_sample_audio_data.shape, "采样率: ", 1000)
        plot_signal(down_sample_audio_data, title=audio_path)
        im = Image.open('/home/pi/Desktop/material/hs_data/hs_img.jpg')
        show_image(im)


        freq1, fre2, bi_spectrum = polycoherence(
            down_sample_audio_data[-2500:], nfft=1024, fs=1000, norm=None, nperse
        bi_spectrum = np.array(abs(bi_spectrum))  # calculate bi_spectrum
        bi_spectrum = bi_spectrum.reshape((256, 256, 1))
        bi_spectrum = 255 * (bi_spectrum - np.min(bi_spectrum)) / \
            (np.max(bi_spectrum) - np.min(bi_spectrum))
        dataset = np.vstack((dataset, np.array([bi_spectrum])))
        dataset = np.delete(dataset, 0, 0)
        dataset = dataset.astype('float32')
    if not GPIO.input(KEY2_PIN):
        show_image(im_testing)
```

```
        interpreter2.allocate_tensors()
        # Get input and output tensors.
        input_details = interpreter2.get_input_details()
        output_details = interpreter2.get_output_details()
        # Test the model on random input data.
        input_shape = input_details[0]['shape']
        interpreter2.set_tensor(input_details[0]['index'], dataset)
        interpreter2.invoke()
        output_data = interpreter2.get_tensor(output_details[0]['index'])[0]
        print(output_data)
        print(np.argmax(output_data))
        print()

        if np.argmax(output_data) == 0:
            show_image(im_normal)
        else:
            show_image(im_abnormal)
    if not GPIO.input(KEY3_PIN):
        show_image(im_testing)
        # Load the TFLite model and allocate tensors.
        # interpreter = tflite.Interpreter(model_path='model.tflite')
        interpreter4 = tf.lite.Interpreter(model_path='/home/pi/Desktop/material/
        interpreter4.allocate_tensors()
        # Get input and output tensors.
        input_details4 = interpreter4.get_input_details()
        output_details4 = interpreter4.get_output_details()
        # Test the model on random input data.
        input_shape = input_details4[0]['shape']
        interpreter4.set_tensor(input_details4[0]['index'], dataset)
        interpreter4.invoke()
        output_data = interpreter4.get_tensor(output_details4[0]['index'])[0]
        print(output_data)
        print(np.argmax(output_data))
        print()
        if np.argmax(output_data) == 0:
            show_image(im_AS)
        elif np.argmax(output_data) == 1:
            show_image(im_MS)
        elif np.argmax(output_data) == 2:
            show_image(im_MR)
        elif np.argmax(output_data) == 3:
            show_image(im_MVP)
```

进而，我们设置树莓派开机时自动运行此python文件，我们打开终端输入：

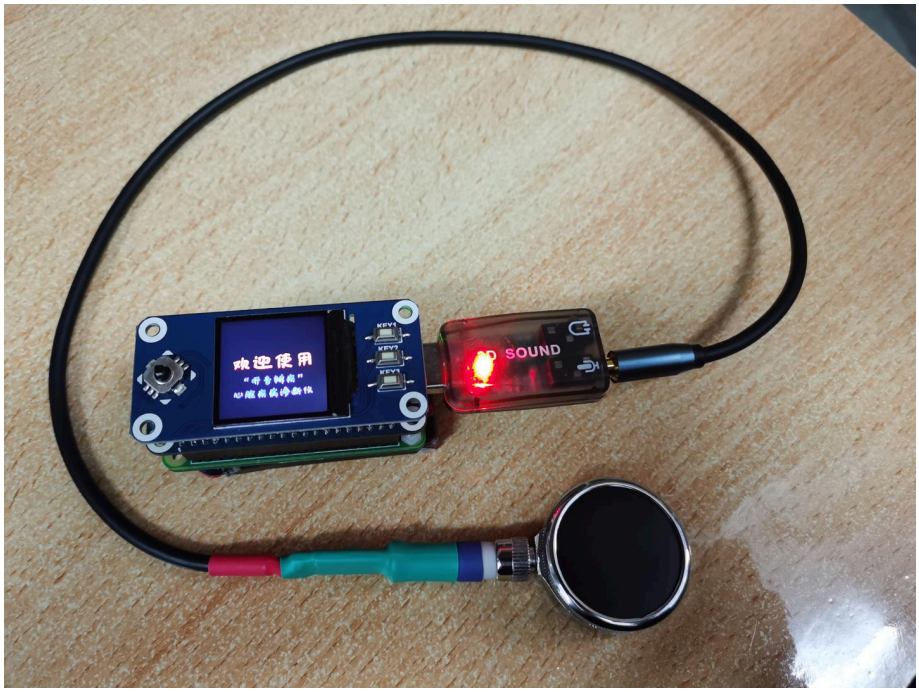
```
sudo nano /etc/rc.local
```

进入文件后，在exit 0之前加入：

```
sleep 1
cd /home/pi

sudo su pi -c "python3 /home/pi/Desktop/raspberry_single_test.py"
```

注：这里得sleep 1可以根据实际情况进行选择是否添加。

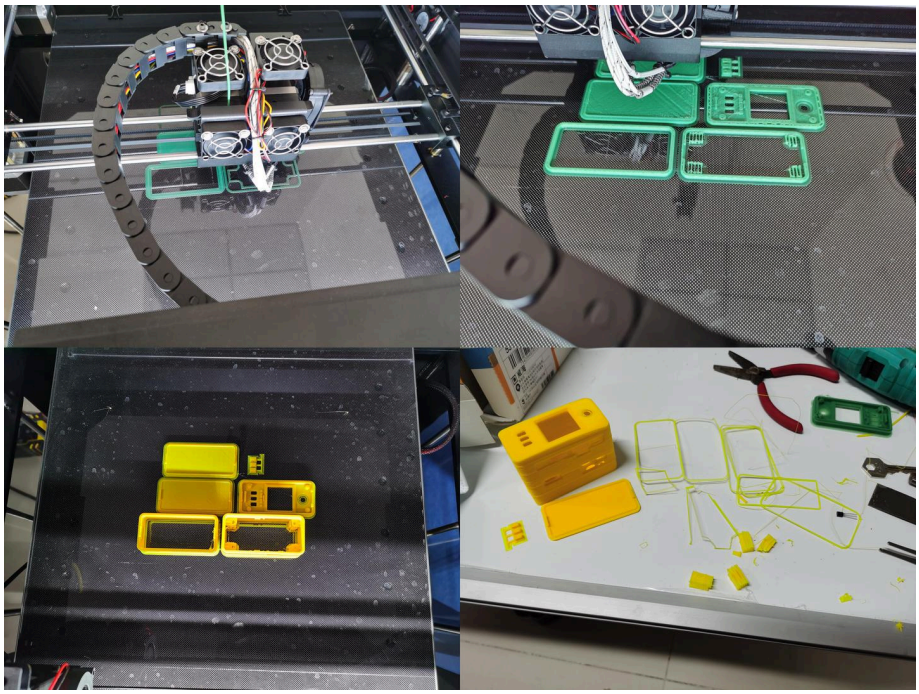


七、美化

这一部分我简介一下树莓派得美化过程。

7.1 硬件美化

上面的的树莓派略显简陋，我们动手制作了一个外壳把它美化一下。我们利用solidworks建模画出诊断仪的外壳，并3D打印出来，展示制作时的一些照片：

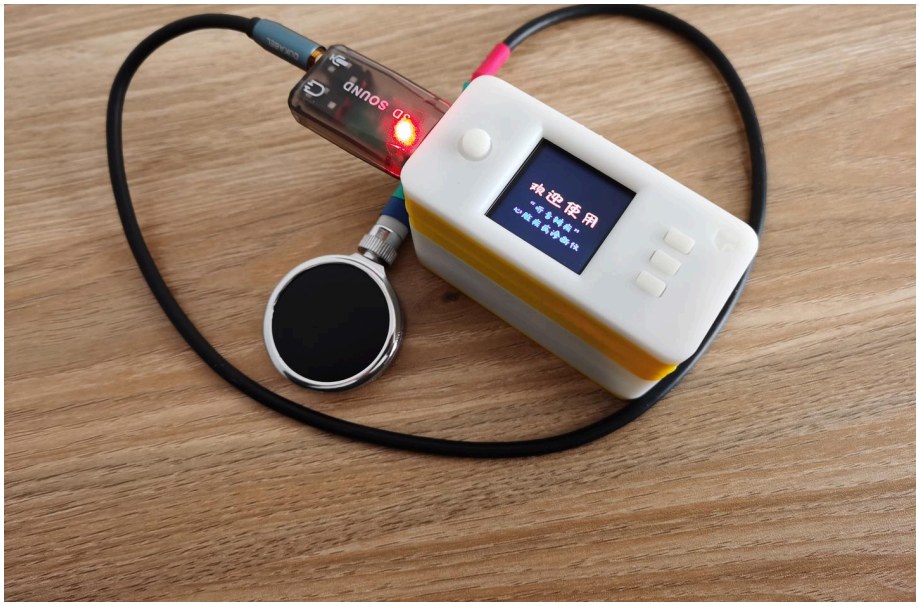


7.2 GUI美化

我们给树莓派的每一个用户界面进行了优化，我们利用创客贴网站绘制用户界面。



至此，我们的树莓派就大功告成啦！晒出我们的最终成果图！



最后声明，这份工作是我大三时候做的，已经不再做相关课题了，但是为了方便各位朋友，相关代码依然公开，本人不负责维护，给出的github repo仅作为纪念意义。同时，正是因为这是我大三时的作业，不可能避免会出现一些问题，大家如果要使用的需要自行矫正，我的内容仅作为参考，也欢迎大家对此改进。谢谢大家的阅读与评论。

送礼物

还没有人送礼物，鼓励一下作者吧

所属专栏 · 2020-12-08 15:18 更新

本科项目

知错就改

2 篇内容 · 184 赞同

订阅

最热内容 · 人体动作识别（HAR）

编辑于 2023-08-18 11:28 · 上海

深度学习（Deep Learning） 树莓派（Raspberry Pi） 医学



豆包大模型，约800万tokens仅需40元，每日限量…

豆包大模型，字节跳动自研大模型，具有更强的推理能力，多模态理解能力，GUI操作能力和前端页面编程能力，更强 … 查看详情

火山引擎 的广告



理性发言，友善互动

72 条评论 默认 最新



winterholiday

拜读了一番，收获颇多，感谢博主

2022-09-11

2021-05-20 回复 2

Jimmy
正在做相关课题的毕设，up真是太棒了
2021-03-20 回复 1

查看全部 18 条回复 ›

Lebron
博主你好，非常感谢你写的这篇文章，对我收获巨大，想问一下关于这个项目你们有发相关文章和专利吗，想拜读一下
2023-08-05 回复 1

为爱改变
你好，我复现您的代码？成功率只有50%多，您的代码是不是没有上传完全呀，能在分享一下吗1759144061 @qq.com ,做毕设使用，感谢
2023-05-12 回复 1

展开其他 5 条回复 ›

Harry
GitHub那个链接好像失效了，能看看文件吗
2022-05-03 回复 1

知错就改 作者
已经更新了，谢谢提醒
2022-05-16 回复 2

我还没秃还能码
楼主，您的模型效果这么好。也许是因为数据集3和数据集4是同一个数据集。。。
2022-04-19 回复 1

004365
你好 请问二阶谱的公式是出自哪篇文章
2022-01-05 回复 1

展开其他 1 条回复 ›

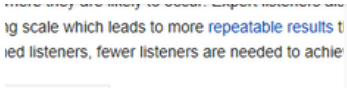
不知道谁
然后想问下您有做数据等长处理吗？因为有的数据小于2.5s怎么办呢
2021-12-27 回复 喜欢

罗松
博主，你好，我在下载第四个链接的数据时电脑提示文件不安全，并阻止了下载，我应该怎么做呢
2025-04-06 回复 喜欢

点击查看全部评论 ›

理性发言，友善互动

推荐阅读



训练有素的听音者与未经训练的听音者之间的区别

鬼斧神工1... 发表于理性派Hi...

声明：平时看些文章做些笔记分享出来，文章中难免存在错误的地方，还望大家海涵。搜集一些资料，方便查阅学习：http://yqli.tech/page/speech.ht... 语音合成领域论文列表请访问...

李永强 发表于AI语音



超声引导神经阻滞，初学者如何才能快速成为老司机（新手必...

陈小捌



10978 Yanny 还是 Laurel？共振峰骗了你

王赟 Maigo